SCV SECURITY

AUDIT REPORT

A10 Capital

Steaming Protocol

Prepared by SCV-Security

On 31st March 2025

Table of Contents

Table of Contents **2**
Introduction **3**
Scope Functionality 3
Submitted Codebase 3
Revisions – Submitted Codebase 3
Methodologies 4
Code Criteria 5
Findings Summary **6**
Findings Technical Details **7**
1. Burning NFTs could cause failures in creating streams 7
2. is_transferable restriction can be bypassed via SendNFT 8
3. update_collection_info and royalties do not implement authentication mechanisms 9
4. Usage of ceil may cause funds to be distributed earlier 10
5. Missing validations when creating streams 11
6. Incorrect event emitted 12
7. Misleading comment 13
Document Control **14**
Appendices **15**
A. Appendix - Risk assessment methodology 15
B. Appendix - Report Disclaimer 16

Introduction

SCV has been engaged by A10 Capital to conduct a comprehensive security review with the goal of identifying potential security threats and vulnerabilities within the codebase. The purpose of this audit is to evaluate the security posture of the codebase and provide actionable recommendations to mitigate any identified risks. This report presents an overview of the findings from our security audit, outlining areas of concern and proposing effective measures to enhance the codebase's security.

Scope Functionality

The streaming protocol features the CW721-streaming and streaming-protocol contracts. The CW721-streaming contracts are NFTs that will be minted by the streaming-protocol contract when users create a stream. Users who hold the NFT can claim rewards based on the shape configurations. Stream owners can cancel streams if `is_cancelable` is set to true, and anyone can deposit funds for `OpenEnded` shapes to continue the distribution.

Submitted Codebase

streaming-protocol	
Repository	https://github.com/A10-Capital/streaming-protocol
Commit	`7d27e623e3fd200a68313ad04199ec34b3bb0ed3`
Branch	`main`

Revisions – Submitted Codebase

streaming-protocol	
Repository	https://github.com/A10-Capital/streaming-protocol
Commit	`2237859b1d5cc98b0fdf2c0b9cac872df6377be2`
Branch	`main`

Methodologies

SCV performs a combination of automated and manual security testing based on the scope of testing. The testing performed is based on the extensive experience and knowledge of the auditor to provide the greatest coverage and value to A10 Capital. Testing includes, but is not limited to, the following:

- Understanding the application and its functionality purpose.
- Deploying SCV in-house tooling to automate dependency analysis and static code review.
- Analyse each line of the code base and inspect application security perimeter.
- Review underlying infrastructure technologies and supply chain security posture.

Code Criteria

This section provides an evaluation of specific criteria aspects as described below:

- **Documentation:** Evaluating the presence and comprehensiveness of publicly available or provided explanatory information, diagram flowcharts, comments, and supporting documents to enhance code understanding.
- **Coverage:** Evaluating whether the code adequately addresses all necessary cases and scenarios, ensuring that the intended functionality or requirements are sufficiently covered.
- **Readability:** Assessing how easily the code can be understood and maintained, considering factors such as code structure, naming conventions, and overall organisation.
- **Complexity:** Evaluating the complexity of the code, including factors such as, number of lines, conditional statements, and nested structures.

The status of each criteria is categorised as either **SUFFICIENT** or **NOT-SUFFICIENT** based on the audit assessment. This categorisation provides insights to identify areas that may require further attention and improvement.

Criteria	Status	Notes
Documentation	**SUFFICIENT**	N/A
Coverage	**SUFFICIENT**	N/A
Readability	**SUFFICIENT**	N/A
Complexity	**SUFFICIENT**	N/A

Findings Summary

Summary Title	Risk Impact	Status
Burning NFTs could cause failures in creating streams	SEVERE	RESOLVED
`is_transferable` restriction can be bypassed via `SendNFT`	SEVERE	RESOLVED
`update_collection_info` and royalties do not implement authentication mechanisms	MODERATE	RESOLVED
Usage of ceil may cause funds to be distributed earlier	MODERATE	RESOLVED
Missing validations when creating streams	LOW	RESOLVED
Incorrect event emitted	INFO	RESOLVED
Misleading comment	INFO	RESOLVED

Findings Technical Details

1. Burning NFTs could cause failures in creating streams

RISK IMPACT: SEVERE	STATUS: RESOLVED

Description

In `contracts/streaming-protocol/src/execute.rs:124-162`, the `create_stream` function mints an NFT based on the `NumTokensResponse.count` value, which represents the total NFT supply.

However, users can purposely burn an NFT to decrease the `NumTokensResponse.count` value, which causes an error when minting new NFTs as the token ID has already been taken.

For example, assume three NFTs were minted with token IDs of `0`, `1`, and `2`. If the owner of token ID `0` burns their NFT with `Cw721ExecuteMsg::Burn`, the total NFT supply will decrease in `packages/cw721/src/execute.rs:307`. If a user tries to create a stream, the `NumTokens` query will return `2`, which causes the `Mint` transaction to fail because token ID `2` is already taken, as seen in `packages/cw721/src/execute.rs:374`.

As a result, a denial of service will occur as streams cannot be created, preventing the protocol from working as intended.

Recommendation

Consider modifying the cw721-streaming contract to disallow `Cw721ExecuteMsg::Burn` from being executed.

2. `is_transferable` restriction can be bypassed via SendNFT

RISK IMPACT: SEVERE	STATUS: RESOLVED

Description

The cw721-streaming contract implements an `is_transferable` validation in `contracts/cw721-streaming/src/lib.rs:86-105` that ensures the NFT can only be transferred if the `is_transferable` attribute is set to true. This field is defined when creating a stream in `contracts/streaming-protocol/src/execute.rs:145`.

However, this restriction only applies to the `TransferNft` message and not `SendNFT`. Users can still transfer NFTs to recipients (e.g., contracts) via the `SendNFT` message, bypassing the `is_transferable` restriction.

Recommendation

Consider enforcing the `is_transferable` validation for `SendNFT` messages.

3. `update_collection_info` and royalties do not implement authentication mechanisms

RISK IMPACT: MODERATE	STATUS: RESOLVED

Description

The `update_collection_info` function in `packages/cw721/src/execute.rs:315` allows updating the extension metadata. This will eventually call the `validate` function in `packages/cw721/src/msg.rs:683`, which is responsible for authentication to ensure the caller is the `CREATOR`.

The issue is that authentication is only performed if the `name` or `symbol` field is supplied without considering the `extension` field, as seen in `packages/cw721/src/msg.rs:701-709`. This means that if an `update_collection_info` request is specified with the `name` or `symbol` field as `None` while the `extension` field is `Some(_)`, the authentication will be skipped, allowing anyone to update the extension for the NFTs.

Similarly, the `CollectionExtension` in `packages/cw721/src/msg.rs:398` does not authenticate if the user specifies the `royalty_info` field to be updated. This is shown in the `validate` traits for `CollectionExtensionMsg` and `RoyaltyInfo`, which can be found in `packages/cw721/src/msg.rs:512` and `packages/cw721/src/msg.rs:558`.

Recommendation

Consider implementing authentications in the `update_collection_info` function and when updating `RoyaltyInfo` in `CollectionExtension`. Test case provided in *Appendix C*.

4. Usage of ceil may cause funds to be distributed earlier

RISK IMPACT: MODERATE	STATUS: RESOLVED

Description

The `calculate_withrawable_amount` function in `contracts/streaming-protocol/src/helpers.rs:157` computes the withdrawal amount allowed based on the elapsed duration.

The issue is that the `to_uint_ceil` function is implemented when computing the withdrawal amount for `Shape::Linear`, `Shape::LinearWithCliff`, and `Shape::OpenEnded`. This is problematic because `to_uint_ceil` will round up the computed amount, potentially resulting in funds being distributed earlier than intended.

Recommendation

Consider replacing `to_uint_ceil` with `to_uint_floor`.

5. Missing validations when creating streams

RISK IMPACT: LOW	STATUS: RESOLVED

Description

The `create_stream` function in `contracts/streaming-protocol/src/execute.rs:38` allows users to create streams with different types of shapes.

These shapes can benefit from further validations as follows:

- In `Shape::LinearWithCliff`, the `cliff_time` variable should be validated to be less than `end_time`.
- In `Shape::OpenEnded`, the `rate` variable should be validated to be less than or equal to 100%.

Recommendation

Consider applying the recommendations mentioned above.

6. Incorrect event emitted

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

In `contracts/streaming-protocol/src/contract.rs:86`, the `owner` attribute is emitted as `info.sender`. This is incorrect because the owner may be `msg.owner` instead of the caller, as seen in line 46.

Recommendation

Consider setting the attribute value to the `owner` variable declared in line 46.

7. Misleading comment

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

In `contracts/streaming-protocol/src/contract.rs:25`, a comment states, "*Initializing the stream ID counter to 0 and saving it to the contract state*". However, this comment is misleading because there is no such implementation.

Recommendation

Consider removing the comment.

Document Control

Version	Date	Notes
-	10th March 2025	Security audit commencement date.
0.1	22nd March 2025	Initial report with identified findings delivered.
0.5	30th March 2025	Fixes remediations implemented and reviewed.
1.0	31st March 2025	Audit completed, final report delivered.

Appendices

A. Appendix – Risk assessment methodology

SCV-Security employs a risk assessment methodology to evaluate vulnerabilities and identified issues. This approach involves the analysis of both the LIKELIHOOD of a security incident occurring and the potential IMPACT if such an incident were to happen. For each vulnerability, SCV-Security calculates a risk level on a scale of 5 to 1, where 5 denotes the highest likelihood or impact. Consequently, an overall risk level is derived from combining these two factors, resulting in a value from 10 to 1, with 10 signifying the most elevated level of security risk

Risk Level	Range
CRITICAL	10
SEVERE	From 9 to 8
MODERATE	From 7 to 6
LOW	From 5 to 4
INFORMATIONAL	From 3 to 1

LIKELIHOOD and **IMPACT** would be individually assessed based on the below:

Rate	LIKELIHOOD	IMPACT
5	**Extremely Likely**	Could result in severe and irreparable consequences.
4	**Likely**	May lead to substantial impact or loss.
3	**Possible**	Could cause partial impact or loss on a wide scale.
2	**Unlikely**	Might cause temporary disruptions or losses.
1	**Rare**	Could have minimal or negligible impact.

B. Appendix – Report Disclaimer

This report should not be regarded as an "endorsement" or "disapproval" of any specific project or team. These reports do not indicate the economics or value of any "product" or "asset" created by a team or project that engages SCV-Security for a security review. The audit report does not make any statements or warranties about the code's utility, safety, suitability of the business model, regulatory compliance of the business model, or any other claims regarding the fitness of the implementation for its purpose or its bug-free status. The audit documentation is intended for discussion purposes only. The content of this audit report is provided "as is," without representations and warranties of any kind, and SCV-Security disclaims any liability for damages arising from or in connection with this audit report. Copyright of this report remains with SCV-Security.

THANK YOU FOR CHOOSING



SCV SECURITY

scv.services

contact@scv.services